

Experimental Evaluation and Optimization of Real-Time Scheduling on STM32

Vladimir Toporkov, Artem Burmyakov
Innopolis University

I. INTRODUCTION

A. Contribution

Our contributions are about evaluation of schedulers for STM32 microcontroller:

- we first implement Super Loop scheduler,
- then we implement the Earliest-Deadline First (EDF) scheduler, and finally
- we evaluate their performance upon a real STM32 hardware.

II. BACKGROUND ON STM32

The experimental platform consists of a uniprocessor STM32F767 microcontroller and external sensors connected through its hardware peripherals, as shown in Fig. 1.

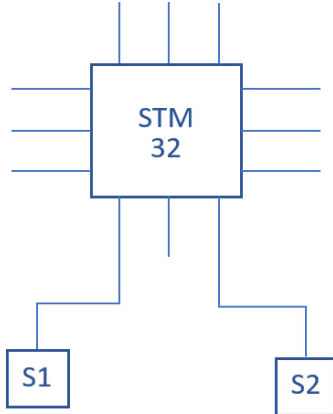


Fig. 1. STM32 hardware platform and connected sensors.

The hardware components are presented in Table I.

TABLE I
EXPERIMENTAL HARDWARE PLATFORM.

Component	Purpose
STM32F767	Scheduling and task execution
DHT11	Temperature and humidity measurement
HC-SR04	Distance measurement
OV2640	Image capturing

III. SYSTEM MODEL

We model the workload by a set of periodic tasks $\mathcal{T} = \{\tau_1, \dots, \tau_n\}$ executed on a uniprocessor. We consider only implicit deadlines, i.e., $D_i = T_i$. Each task $\tau_i = (C_i, T_i = D_i)$ generates a sequence of jobs J_i^j , where j denotes the scheduled release index. Consecutive scheduled releases satisfy $R_i^{j+1} = R_i^j + T_i$.

TABLE II
SYSTEM MODEL NOTATION.

Symbol	Meaning
$\mathcal{T}, n$	Task set and number of tasks
τ_i	Task i
C_i	Execution time of a job of τ_i
T_i	Task period
$D_i = T_i$	Relative deadline, equal to the period
J_i^j	Job associated with the j -th scheduled release of τ_i
R_i^j	Scheduled release time
f_i^j	Completion time of an executed job
$d_i^j = R_i^j + D_i$	Absolute deadline
$U = \sum_{i=1}^n C_i/T_i$	Task-set utilization

Table II summarizes the notation. Release indices include skipped activations, for which no completion time is defined.

- A *schedule* specifies which job executes on the processor at each time instant, or whether the processor executes no job.
- *Preemption* interrupts a job before completion to execute another job. The interrupted job may resume later.
- A completed job J_i^j has missed its deadline if $f_i^j > d_i^j$.

IV. SCHEDULING PRINCIPLES

A. Super Loop Scheduler

The considered Super Loop scheduler repeatedly scans tasks in a fixed order and executes each selected job non-preemptively. To illustrate its behavior, let us consider the two periodic tasks in Table III.

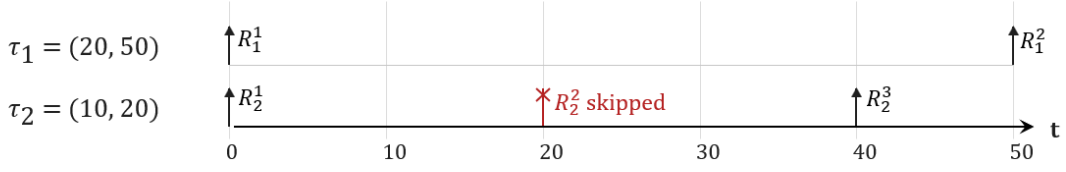
TABLE III
TASK SET USED IN THE SUPER LOOP EXAMPLE.

Task	C_i	T_i	D_i
τ_1	20	50	50
τ_2	10	20	20

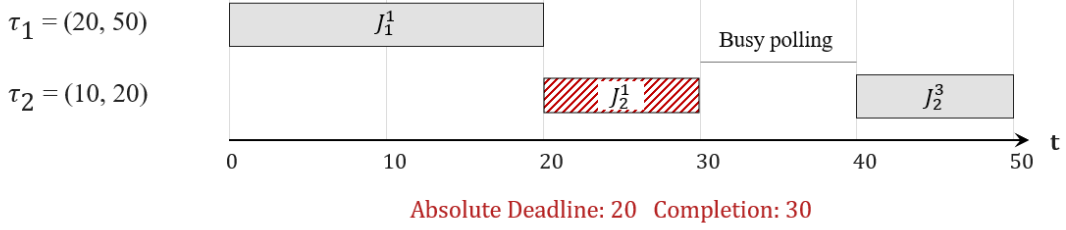
The resulting Super Loop schedule is shown in Fig. 2.

$$\tau_i = (C_i, D_i = T_i)$$

Scheduled Releases



Super Loop execution scenario



J_i^j – job execution R_i^j – scheduled release time

Fig. 2. Super Loop execution scenario for the task set in Table III.

The corresponding scheduling state is shown in Table IV.

Let t^{cur} denote the current system time and t_i^{next} the next scheduled release time of task τ_i . During execution of J_i^j , the release variable t_i^{next} retains R_i^j . At completion, $t^{cur} = f_i^j$, and a deadline miss is recorded if $f_i^j > d_i^j$. The scheduler then advances t_i^{next} by T_i . While $t_i^{next} < t^{cur}$, that release is counted as skipped and t_i^{next} is advanced by T_i again. A release at $t_i^{next} = t^{cur}$ is retained.

TABLE IV
SUPER LOOP SCHEDULING STATE FOR FIG. 2. RELEASE TIMES ARE SHOWN AFTER THE CHECKS AT EACH INSTANT.

t^{cur}	t_1^{next}	t_2^{next}	Deadline miss check	Release skip check
0	0	0	–	–
20	50	0	$20 < R_1^1 + 50$	$20 < R_1^2$
30	50	40	$30 > R_2^1 + 20$	$30 > R_2^2$ $30 < R_2^3$
40	50	40	–	–
50	50	60	$50 < R_2^3 + 20$	$50 < R_2^4$

In Fig. 2, J_1^1 executes from $t = 0$ to $t = 20$. Then, J_2^1 executes until $t = 30$, missing its deadline at $t = 20$. At completion, the release scheduled for $R_2^2 = 20$ is counted as skipped and t_2^{next} advances to $R_2^3 = 40$. Thus, no J_2^2 is executed.

The scheduling logic illustrated in Fig. 2 is summarized in Algorithm 1.

B. Earliest Deadline First

1.AB2VT: To describe pseudo-code of EDF

Algorithm 1 Super Loop Scheduler

```

1: function SUPERLOOP( $\mathcal{T}$ )
2:   for all  $\tau_i \in \mathcal{T}$  do
3:      $t_i^{next} \leftarrow 0$ 
4:   end for
5:   while true do
6:     for  $i \leftarrow 1$  to  $n$  do ▷ Fixed task order
7:        $t^{cur} \leftarrow \text{Now}$ 
8:       if  $t^{cur} \geq t_i^{next}$  then
9:          $R_i^j \leftarrow t_i^{next}$ 
10:        EXECUTE( $J_i^j$ ) ▷ Non-preemptive
11:         $f_i^j \leftarrow \text{Now}$ 
12:        if  $f_i^j > R_i^j + D_i$  then
13:          RECORDMISS( $J_i^j$ )
14:        end if
15:         $t_i^{next} \leftarrow t_i^{next} + T_i$ 
16:        while  $t_i^{next} < f_i^j$  do
17:          RECORDSKIP( $t_i^{next}$ )
18:           $t_i^{next} \leftarrow t_i^{next} + T_i$ 
19:        end while
20:      end if
21:    end for
22:  end while
23: end function

```

The Super Loop mechanism does not take deadlines into account, jobs are selected according to a fixed loop order rather than their deadlines. As shown in Fig. 2, this may delay jobs beyond their deadlines. The considered implementation skips

releases according to the rule above.

We consider preemptive EDF, which dynamically selects the ready job with the earliest absolute deadline d_i^j (Section III).

If a newly released job has an earlier deadline than the running job, the running job is preempted.

V. PERFORMANCE EVALUATION

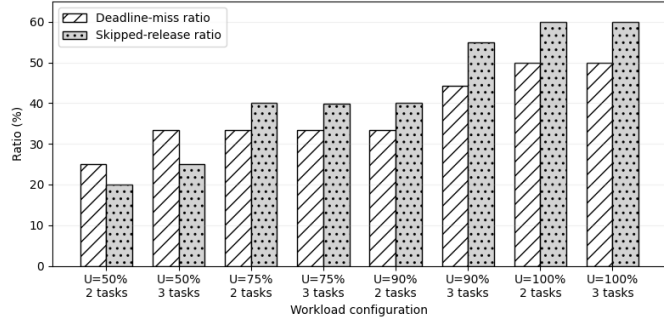


Fig. 3. Per-task deadline-miss and skipped-release ratios under Super Loop. Each configuration specifies the task-set utilization U and the number of tasks.